

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I M.veeranjanya Reddy(192472140)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.veeranjanya Reddy

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints: A. Limited processing power (mobile CPU)

B. Real-time performance

C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

Ans:

The system must detect **happy, sad, and neutral** expressions while using limited mobile CPU, providing real-time performance, and handling slight head movement and lighting changes.

Proposed Pipeline

Camera Input → Preprocessing → Face Detection → Face ROI → Feature Extraction → Expression Classification → Output

1. **Input Acquisition:**

Capture live frames using the mobile camera through OpenCV.

2. **Preprocessing:**

Resize frames to a smaller resolution to reduce CPU usage. Convert frames to grayscale and use histogram equalization/CLAHE to reduce the effect of lighting changes.

3. **Face Detection:**

Use OpenCV's lightweight **Haar Cascade face detector** to locate the face. Only the detected face region is processed further.

4. **Face ROI and Alignment:**

Crop the face and resize it to a fixed size. Eye/landmark positions can be used to compensate for small head rotations.

5. **Expression Analysis:**

Extract lightweight features such as **LBP or HOG** from the face region. A lightweight **SVM classifier** classifies the expression as:

- Happy
- Sad
- Neutral

6. **Temporal Smoothing:**

Use predictions from consecutive frames and majority voting to avoid sudden incorrect expression changes.

7. **Final Output:**

Display the detected expression and confidence on the mobile screen.

Justification

- **Limited CPU:** Low-resolution images and lightweight Haar + LBP/HOG + SVM reduce computation.
- **Real-time:** Only the face ROI is processed instead of the complete image.
- **Head movement:** Face detection and alignment handle slight movement.
- **Lighting changes:** Grayscale and histogram equalization improve robustness.
- **Accuracy:** Temporal smoothing reduces incorrect frame-level predictions.

Final pipeline:

Mobile Camera



Resize + Grayscale



Lighting Normalization



Haar Face Detection



Face ROI + Alignment



LBP/HOG Features



SVM Classifier



Happy / Sad / Neutral



Display Result

2. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Ans:

The system must **detect vehicles, track them across frames, count vehicles, display statistics, and operate continuously with minimal delay.**

Proposed Pipeline

Live Video → Frame Processing → Vehicle Detection → Tracking → Counting → Statistics → Visualization

1. Video Handling:

Capture live CCTV/video using OpenCV's VideoCapture().

2. Preprocessing:

Resize frames to reduce processing time and apply Gaussian blur to reduce noise.

3. Vehicle Detection:

For a fixed traffic camera, use **background subtraction such as MOG2** to identify moving objects. Morphological operations remove noise, and contours are used to obtain vehicle bounding boxes.

4. Vehicle Tracking:

Calculate the centroid of every detected vehicle and assign a unique ID. Match the centroid with the previous frame to track the same vehicle.

5. Vehicle Counting:

Draw a virtual counting line on the road. When a tracked vehicle crosses the line, increase the vehicle count. The vehicle ID prevents the same vehicle from being counted

multiple times.

6. **Statistics:**

Maintain values such as:

Cars : 120
Buses : 15
Trucks : 10
Total : 145

7. **Visualization:**

Use OpenCV drawing functions:

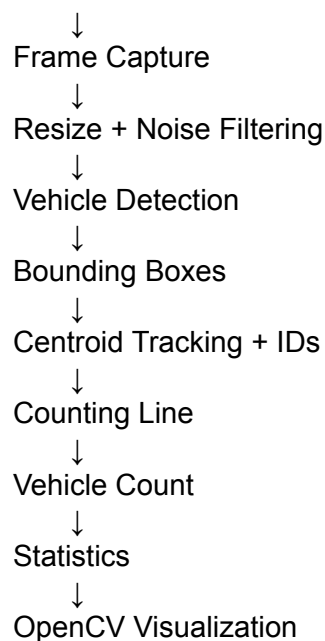
- cv2.rectangle() for vehicle boxes
- cv2.putText() for IDs and counts
- cv2.line() for the counting line
- cv2.circle() for vehicle centroids

Justification

- **Accuracy:** Tracking IDs prevent duplicate counting.
- **Real-time performance:** Resize frames and use lightweight detection.
- **Minimal delay:** Process frames directly without unnecessary storage.
- **Noise reduction:** Morphological operations and contour-area filtering remove small unwanted objects.
- **Continuous operation:** Frames are processed continuously from the live stream.

Final pipeline:

CCTV / Live Video



3. Real-Time Object Detection on Mobile Device

A mobile application must:

- A. Detect objects using the camera
- B. Run in real time
- C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how

your design ensures efficiency under constraints.

Ans:

The system must detect objects using the camera, operate in real time, and use limited CPU and memory.

Proposed Pipeline

Camera → Frame Optimization → Preprocessing → Object Detection → Filtering → Display

1. **Image Acquisition:**
Capture frames from the mobile camera using OpenCV.
2. **Frame Optimization:**
Resize frames to a smaller resolution such as 320×240 or 640×480. If required, process alternate frames to reduce CPU usage.
3. **Preprocessing:**
Normalize the image and convert it into a blob using:
`cv2.dnn.blobFromImage()`
4. **Object Detection:**
Use a **lightweight deep-learning model through OpenCV DNN**, such as a compact YOLO-based model, to detect objects.
5. **Confidence Filtering:**
Remove detections whose confidence is below a selected threshold, for example 0.5.
6. **Non-Maximum Suppression:**
Use `cv2.dnn.NMSBoxes()` to remove multiple overlapping boxes for the same object.
7. **Display:**
Draw bounding boxes and labels using `cv2.rectangle()` and `cv2.putText()`.

Example:

Person - 94%

Bottle - 87%

Car - 91%

Justification

- **Limited CPU:** Use a lightweight detection model.
- **Limited memory:** Use small input resolution and avoid storing unnecessary frames.
- **Real-time:** Frame resizing and frame skipping reduce processing time.
- **Accuracy:** Confidence threshold and NMS remove unreliable and duplicate detections.

Final pipeline:

Mobile Camera



Frame Capture



Resize



Normalization



Blob Creation



Lightweight OpenCV DNN



Confidence Filtering



NMS



Bounding Boxes + Labels



Mobile Display

4. Smart Attendance System using Face Recognition

A system must:

- A. Capture faces from a webcam
- B. Recognize registered users
- C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

ANS:

The system must **capture faces, recognize registered users, mark attendance automatically, and avoid false recognition.**

Proposed Pipeline

Webcam → Face Detection → Preprocessing → Face Recognition → Verification → Attendance → Database

1. **Face Capture:**
Capture live video from the webcam using OpenCV.
2. **Face Detection:**
Use Haar Cascade or a lightweight OpenCV DNN detector to detect faces in each frame.
3. **Preprocessing:**
Crop the detected face, resize it to a standard size, convert it to grayscale, and normalize lighting.
4. **Face Recognition:**
Use OpenCV's **LBPH face recognizer** to compare the detected face with registered users.
5. **False Recognition Prevention:**
Apply a recognition threshold. If the match is not sufficiently strong, classify the person as **Unknown**.

Also require recognition in multiple consecutive frames before marking attendance.

6. **Attendance:**
After successful verification, record:

Name | Date | Time | Status

Vijay | 24/08/26 | 09:05 | Present

7. **Duplicate Prevention:**
Check whether the user's attendance has already been recorded for the current session. If yes, do not create another entry.
8. **Output:**
Display the person's name and attendance status on the screen.

Justification

- **False recognition:** Confidence threshold + multiple-frame verification.
- **Lighting variation:** Image normalization.
- **Efficiency:** LBPH is lightweight and suitable for a basic OpenCV system.
- **Duplicate attendance:** Maintain a list/database of already marked users.
- **Unknown users:** Reject matches below the threshold.

Final pipeline:

Webcam



Face Detection



Face Cropping



Resize + Grayscale + Normalization



LBPH Face Recognition



Confidence/Threshold Check



Multi-frame Verification



Known User?



YES



NO



Attendance



Unknown



Database



Display

5. Motion Detection for Home Security

A home security system must:

- A. Detect motion from a camera feed
- B. Trigger alerts only for significant movement
- C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

Ans:

The system must detect motion, trigger alerts only for significant movement, and reduce false alarms caused by noise and background variations.

Proposed Pipeline

Camera → Preprocessing → Background Subtraction → Noise Removal → Motion Detection → Significance Check → Alert

1. **Camera Input:**
Capture continuous frames from the security camera using OpenCV.
2. **Preprocessing:**
Resize the frame, convert it to grayscale, and apply Gaussian blur to reduce camera noise.
3. **Background Modeling:**
Use OpenCV's **MOG2 background subtraction** to separate moving objects from the background.
4. **Noise Removal:**
Apply morphological opening and closing to remove small noise and fill gaps in detected regions.

5. **Motion Detection:**

Use cv2.findContours() to identify motion regions. Calculate the contour area and ignore regions below a minimum area threshold.

6. **Significant Movement:**

Do not trigger an alert for a single-frame change. Require the motion to remain present for several consecutive frames.

7. **Alert:**

If significant motion is confirmed, trigger an alert and optionally save the frame/video.

8. **Visualization:**

Draw a bounding box around the moving object and display:

MOTION DETECTED

Justification

- **Noise:** Gaussian blur removes small image noise.
- **Background variations:** MOG2 continuously models the background.
- **False alarms:** Minimum contour area removes small movements.
- **Temporary changes:** Consecutive-frame verification prevents one-frame alerts.
- **Continuous monitoring:** Frames are continuously processed from the camera.

Final pipeline:

Security Camera



Frame Capture



Resize + Grayscale



Gaussian Blur



MOG2 Background Subtraction



Morphological Filtering



Contour Detection



Area Threshold



Consecutive Frame Check



Significant Motion?

↓
NO

↓
YES

↓
Continue

↓
Trigger Alert

↓
Save + Display

↓
Save + Display